

CPT102 Final Exam Guideline

Introduction to data structure

Example Question 1: Why do we need different types of data structures? Can we not just use **a specific structure** for everything? [Review Lecture 1 5/31]

* If you wish, you can replace the **placeholder** with any possible structure like list, array, tree. The main purpose of this question is to test your understanding of the limitation of a structure and other structure is essential, why?

Example Question 2: which of the following process can be described as an *algorithm*?

- A. a process that are not clear
- B. an infinite sequence manipulation
- C. a description without the output
- D. the correct answer

In general, such question require you have the ability to understand the algorithm definition [Review Lecture 2 3/39].

Example Question 3: Write a pseudocode based on a defined algorithm. For example, quick sort algorithm [Review Lecture 2 12/39].

* Some of you may worry about the format of pseudocode, because you also take INT102. The pseudocode is quite different in these two modules but please bear in mind: CPT102 pseudocode just need to mimic good code and English, and the teacher will understand your behaviors ^_^

Example Question 4: Time complexities, here I give you some mock questions:

```
int pl(int n) {  
    int count = 0;  
    for (int i = 1; i <= n; i *= 2) {  
        for (int j = 1; j <= n; j += i) {  
            for (int k = 1; k <= j; k *= 2) {  
                count++;  
            }  
        }  
    }  
    return count;  
}
```

```
int p2(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i; j <= n; j += i) {
            for (int k = j; k > 0; k /= 2) {
                count++;
            }
        }
    }
    return count;
}
```

```
int p3(int n) {
    int count = 0;
    for (int i = n; i > 0; i /= 2) {
        for (int j = 0; j < n; j += i) {
            for (int k = 0; k < i; k++) {
                count++;
            }
        }
    }
    return count;
}
```

```
int p4(int n) {
    if (n <= 1) {
        return 1;
    }
    int count = 0;
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 0; j < n / i; j++) {
            count++;
        }
    }
    return p4(n / 2) + count;
}
```

```
int p5(int n) {
    if (n <= 1) {
        return 1;
    }
    int count = 0;
    for (int i = 0; i < n; i++) {
        int j = i + 1;
        while (j < n) {
            count++;
            j *= 2;
        }
    }
    return count;
}
```

```
int p7(int n) {
    int count = 0;
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 1; j <= n; j++) {
            for (int k = 1; k <= n; k += j) {
                count++;
            }
        }
    }
    return count;
}
```

```
int p8(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            int k = n;
            while (k > 0) {
                count++;
                k -= j;
            }
        }
    }
    return count;
}
```

```
int p9(int n) {
    if (n <= 1) {
        return 1;
    }
    int count = 0;
    for (int i = 1; i <= n; i *= 2) {
        count += n;
    }
    return p9(n / 2) + p9(n / 4) + count;
}
```

```
int p10(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n / i; j++) {
            for (int k = 1; k <= i * j; k *= 2) {
                count++;
            }
        }
    }
    return count;
}
```

* p6 is not showed for other secure reasons. Sorry for that. If you complete that, you can share your answer in the comment box in Zhitiao, I will tell you the answer. Again, I'm not here to teach you something, if you have any question, you can ask your teacher.

Data Structure - ADT

Before this section, I would like to remind you to review the following concepts of data structures, which may help you gain an overview of how data structures are organized and represented.

A specific data structure can be understood from both the *logical structure* and the *physical structure* perspectives.

The *logical structure* describes the relationships between data elements. It mainly includes:

* *Linear structure*: a one-to-one sequential relationship, where each element (except the first and last) has exactly one predecessor and one successor. Typical examples include arrays, linked lists, stacks, and queues.

* *Tree structure*: a one-to-many hierarchical relationship, where each node has only one predecessor (parent node) but may have multiple successor nodes (children nodes). Typical examples include binary trees, heaps, and general trees.

* *Graph structure*: a many-to-many relationship, where a node may be connected to multiple other nodes simultaneously. Unlike tree structures, graph structures allow more complex connections and cycles. Typical examples include social networks, maps, and network routing systems.

The *physical structure* describes how data elements are stored in computer memory. It mainly includes:

* *Sequential storage structure*: data elements are stored in continuous memory locations. This makes it easy to access elements by index, such as in arrays.

* *Linked storage structure*: data elements are stored in separate memory locations, and each element uses pointers or references to connect to other elements. A typical example is a linked list.

* *Indexed storage structure*: an additional index table is used to record the storage location of data elements, which helps improve search efficiency.

* *Hash storage structure*: a hash function is used to calculate the storage address of each data element, allowing fast insertion, deletion, and search operations.

Example Question 4: Demonstrate your understanding on a specific structure. What is the pros and cons of xxx structure? Compared with xxx structure, what is the advantages/weakness of xxx structure?

Data Structure	Strengths	Weaknesses	Suitable Scenarios
Array			
Linked List			
Stack			
Queue			
Tree			
Graph			
Hash Table			

* ps, please filling the table to help you review the content.

Sort & Search

Example Question 5: sorting & searching algorithm, which is the possible sequence of quick /bubble/insertion/selection/merge/counting/radix sort, does it stable/in-place, why? Which is the possible search sequence of binary search? The complexities of sort algorithm [See lecture 6: 29/30]?

Example Question 6: A quick sort algorithm is sorting a list in ascending order.

In each partition, the first element of the current sublist is selected as the pivot. Suppose that after the first partition, the whole list is divided around the value 50. Then quick sort continues to partition the left sublist around the value 30. Which of the following sequences could be a possible intermediate result after these two partition operations?

Option	Sequence
A	[20, 10, 25, 30, 45, 55, 40, 50, 70, 35, 60]
B	[20, 10, 35, 30, 25, 45, 40, 50, 70, 55, 60]
C	[20, 10, 25, 30, 45, 35, 40, 50, 70, 55, 60]
D	[10, 20, 25, 30, 35, 40, 45, 55, 50, 70, 60]
E	[20, 25, 10, 30, 35, 45, 55, 50, 60, 40, 70]

Example Question 7: A binary search algorithm is searching for the target value 67 in an ascending sorted list (The target value 67 maybe not in the list). Which of the following sequences could be a possible sequence of values checked during the search?

Option	Checked Value Sequence
A	50 → 80 → 65 → 72 → 68
B	50 → 80 → 65 → 72 → 60 → 67
C	50 → 80 → 65 → 72 → 75 → 67
D	50 → 80 → 65 → 72 → 68 → 66
E	50 → 80 → 65 → 72 → 68 → 69 → 67

Tree Algorithm

Since the tree lecture slide is not so well-organized, not so helpful for reviewing. You may double check with the following.

Knowledge Point	Brief Explanation
BST / Binary Search Tree	A binary tree where the left child is smaller than the parent node, and the right child is larger than the parent node. It supports efficient searching, insertion, and deletion if the tree is balanced.
AVL Tree	A self-balancing binary search tree. It automatically adjusts its structure after insertion or deletion to keep the tree balanced, so searching remains efficient.
Heap	A complete binary tree used to quickly access the maximum or minimum element. In a max-heap, each parent node is greater than or equal to its children; in a min-heap, each parent node is smaller than or equal to its children.

Example Question 8: A Binary Search Tree is created by inserting the following values in order: 50, 30, 70, 20, 40, 60, 80, 35, 45, 65
Then the value 30 is deleted from the tree.

Assume that when deleting a node with two children, the node is replaced by its in-order successor.

Questions:

1. Draw the BST before deleting 30.
2. Draw the BST after deleting 30.
3. Write the in-order traversal after deletion.

Example Question 9: AVL Tree Rotation

The following values are inserted into an empty AVL tree in order: 30, 10, 20, 40, 50, 25

Questions:

1. Draw the AVL tree after all insertions.
2. Identify which rotations are required during the insertion process.
3. Explain why AVL trees are more efficient than ordinary BSTs in the worst case.

Example Question 10: Min-Heap Operation

A min-heap is stored as an array: [5, 12, 8, 20, 15, 10, 18, 30, 25]

Then the following operations are performed:

insert 6

deleteMin
insert 4

Questions:

1. Write the heap array after inserting 6.
2. Write the heap array after deleting the minimum value.
3. Write the final heap array after inserting 4.
4. Explain why the heap is not necessarily sorted.

Graphs Algorithm

Some terminology here:

Term	Meaning
Vertex / Node	A point in the graph
Edge	A connection between two vertices
Directed Graph	Edges have directions
Undirected Graph	Edges have no directions
Weighted Graph	Each edge has a weight or cost
Unweighted Graph	Edges have no weight

Representa tion	Description	Suitable For
Adjacency Matrix	Uses a 2D array to show whether two vertices are connected	Dense graphs
Adjacency List	Each vertex stores a list of connected vertices	Sparse graphs
Edge List	Stores all edges as pairs or triples	Algorithms that process edges directly

Trave rsal	Main Idea	Common Use
BFS	Visits nodes level by level using a queue	Shortest path in unweighted graphs
DFS	Goes as deep as possible using recursion or a stack	Cycle detection, connectivity, topological sorting

Algorithm	Suitable For
BFS	Unweighted graphs
Dijkstra's Algorithm	Weighted graphs with non-negative edge weights
Bellman-Ford Algorithm	Weighted graphs that may contain negative edges
Floyd-Warshall Algorithm	Shortest paths between all pairs of vertices

Concept	Meaning
Connected Graph	Every vertex can reach every other vertex in an undirected graph
Connected Component	A group of vertices connected to each other
Strongly Connected Graph	In a directed graph, every vertex can reach every other vertex
Weakly Connected Graph	A directed graph becomes connected if directions are ignored

Algorithm	Main Idea
Kahn's Algorithm	Repeatedly remove vertices with in-degree 0
DFS-based Topological Sort	Add each vertex after visiting all its descendants

Algorithm	Main Idea
Kruskal's Algorithm	Sort edges by weight and add the smallest edge that does not form a cycle
Prim's Algorithm	Start from one vertex and repeatedly add the cheapest edge connecting to a new vertex

Example Question 11: Graph Representations and Traversals

Question 1: BFS and DFS Traversal

Consider the following directed graph:

$A \rightarrow B, C$
 $B \rightarrow D, E$
 $C \rightarrow F$
 $D \rightarrow C, G$
 $E \rightarrow D, G$
 $F \rightarrow E$
 $G \rightarrow F$

Starting from vertex A, assume that when there are multiple choices, vertices are visited in alphabetical order.

Which of the following gives the correct BFS and DFS visiting orders?

Option	BFS Order	DFS Order
A	A, B, C, D, E, F, G	A, B, D, C, F, E, G
B	A, B, C, D, E, G, F	A, B, E, G, F, D, C
C	A, C, B, F, D, E, G	A, C, F, E, D, G, B
D	A, B, D, C, F, E, G	A, B, C, F, E, D, G
E	A, B, C, F, D, E, G	A, B, D, G, F, E, C

Example Question 12: Graph Representation Choice

A graph has 10,000 vertices and only 15,000 edges.

Answer the following questions:

1. Is this graph sparse or dense?
2. Which representation is more suitable: adjacency matrix or adjacency list?
3. If we need to frequently check whether an edge exists between two given vertices, which representation would be faster?
4. Explain the trade-off between memory usage and edge lookup speed.

Example Question 13: Dijkstra's Algorithm

Consider the following weighted directed graph:

A → B: 6

A → C: 2

C → B: 1

B → D: 2

C → D: 5

C → E: 4

E → D: 1

D → F: 3

E → F: 7

Starting from vertex A, use Dijkstra's algorithm to find the shortest path from A to F.

Answer the following:

1. What is the shortest distance from A to F?
2. What is the shortest path?
3. Which node should be finalized immediately after A?

Example Question 14: Possible Shortest Path Sequence

A shortest path algorithm is running on a weighted graph with non-negative edge weights.

Suppose the algorithm starts from S and finalizes nodes in the following order:

S, B, A, D, C, E

Which of the following distance sequences could be possible?

Option	Finalized Distance Sequence
A	0, 3, 5, 6, 9, 12
B	0, 3, 2, 6, 9, 12
C	0, 4, 7, 6, 8, 10
D	0, 5, 7, 9, 8, 12
E	0, 2, 6, 10, 9, 8

Example Question 15: Strong Connectivity

Consider the following directed graph:

A → B

B → A

B → C

$C \rightarrow D$
 $D \rightarrow C$
 $D \rightarrow E$
 $E \rightarrow C$
 $F \rightarrow E$
 $F \rightarrow G$
 $G \rightarrow F$

Answer the following questions:

1. Is the graph strongly connected?
2. Is the graph weakly connected?
3. Identify all strongly connected components.
4. Draw or describe the component graph after compressing each strongly connected component into one node.

Example Question 16: Topological Sorting

A directed graph has the following edges:

$A \rightarrow D$
 $B \rightarrow D$
 $B \rightarrow E$
 $C \rightarrow E$
 $D \rightarrow F$
 $E \rightarrow F$
 $E \rightarrow G$
 $F \rightarrow H$
 $G \rightarrow H$

Which of the following are valid topological orderings?

Option	Ordering
A	A, B, C, D, E, F, G, H
B	C, A, B, E, D, G, F, H
C	A, C, D, B, E, F, G, H
D	B, C, E, A, D, G, F, H
E	A, B, D, C, E, F, H, G

Example Question 17: Topological Sorting and Cycles

Consider the following directed graph:

$A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow D$

$C \rightarrow D$

$D \rightarrow E$

$E \rightarrow C$

Answer the following questions:

1. Can this graph have a topological ordering?
2. If not, identify the cycle.
3. Which single edge could be removed to make topological sorting possible?

Example Question 18: Minimum Spanning Tree Using Kruskal's Algorithm

Consider the following weighted undirected graph:

A-B: 4

A-C: 3

B-C: 1

B-D: 2

C-D: 4

C-E: 5

D-E: 1

D-F: 7

E-F: 3

B-E: 6

Using Kruskal's algorithm, answer the following:

1. Which edges are selected in the minimum spanning tree?
2. What is the total weight?
3. Which edge is skipped first because it would create a cycle?
4. Is the minimum spanning tree unique? Explain briefly.

This is the end of this document, hope this can help you in the reading week. We will have another mock test document in the reading week.

If you have finished this question, you can share your answer in Zhitiao!